ISIMG

institut Supérieur de l'informatique et multimédia de Gabès

DOCUMENTATION TECHNIQUE

Mini-Projet — Programmation Orientée Objet

Système de Gestion d'un Restaurant

Application Java Swing · Architecture MVC + DAO · Base de données MySQL

Module	Programmation Orientée Objet
Filière	Cycle Préparatoire Intégré — MPI/MI
Niveau	CPI 2 — Groupe A
Enseignant	M. Sofiane HACHICHA
Encadrante TP	Mme. Nesrine AKROUT
Année univ.	2025 — 2026

1. Introduction

Ce document présente la documentation technique complète du système de gestion d'un restaurant développé dans le cadre du mini-projet du module Programmation Orientée Objet (POO) en CPI 2 à l'ISIMG Gabès.

L'application permet la gestion complète des menus, des plats et des commandes d'un restaurant, en distinguant trois profils d'utilisateurs : le client, la serveuse et le cuisinier. Chaque acteur dispose d'un espace dédié avec des fonctionnalités spécifiques accessibles après authentification.

1.1 Objectifs du projet

- Développer une application Java avec interface graphique Swing
- Connecter l'application à une base de données MySQL
- Appliquer l'architecture MVC (Modèle — Vue — Contrôleur)
- Implémenter le patron de conception DAO pour l'accès aux données
- Respecter les principes de la POO : héritage, polymorphisme, encapsulation

2. Technologies utilisées

Technologie	Version	Rôle
Java	JDK 17+	Langage principal
Java Swing	—	Interface graphique
MySQL	8.0+	Base de données relationnelle
MySQL Connector/J	8.0+	Driver JDBC
IntelliJ IDEA / Eclipse	—	Environnement de développement
draw.io	—	Conception UML
Git	—	Gestion de version

3. Architecture du projet

Le projet respecte une architecture en 4 couches séparées, conformément au patron MVC + DAO :

```
RestaurantApp/
■■■ src/
■   ■■■ util/          DBConnection.java     ← Singleton connexion MySQL
■   ■■■ model/         Entités métier (POJOs)
■   ■   ■■■ Utilisateur.java  ← Classe abstraite
■   ■   ■■■ Client.java / Serveuse.java / Cuisinier.java
■   ■   ■■■ Menu.java / Plat.java
■   ■   ■■■ Commande.java / LigneCommande.java / Facture.java
```

```
■   ■■■ dao/              Accès base de données (JDBC)
■   ■   ■■■ IDao.java          ← Interface générique CRUD
■   ■   ■■■ UtilisateurDAO.java / PlatDAO.java / CommandeDAO.java ...
■   ■■■ controller/       Logique métier
■   ■   ■■■ AuthController.java / CommandeController.java ...
■   ■■■ view/             Interfaces graphiques Swing
■       ■■■ LoginFrame.java    ← Écran de connexion
■       ■■■ Dashboard[Client|Serveuse|Cuisinier]Frame.java
■       ■■■ GestionMenuFrame.java   ← CRUD plats (5 boutons)
■■■ lib/  mysql-connector-j.jar
■■■ sql/  restaurant_db.sql
```

3.1 Flux de données

```
Vue (Swing)  →  Contrôleur  →  DAO  →  Base de données MySQL
                    ↑                        ↓
                  Modèle  ←■■■■■■■■■■■■■■■■■■■■■■■■
```

4. Conception UML

4.1 Diagramme de cas d'utilisation

Le système identifie 3 acteurs principaux interagissant avec le système via des cas d'utilisation distincts selon leur rôle.

Acteur	Cas d'utilisation
Tous	Se connecter au système
Client	Parcourir le menu, Commander des plats
Serveuse	Parcourir le menu, Commander, Voir commandes en cours, Voir commandes reçues, Générer une facture
Cuisinier	Gérer les plats (CRUD), Commencer/Annuler commande, Notifier la serveuse

4.2 Relations UML

- **Commander** <<include>> **Parcourir le menu** : impossible de commander sans consulter le menu
- **Notifier la serveuse** <<include>> **Commencer/Annuler commande** : notification déclenchée automatiquement

4.3 Diagramme de classes — Hiérarchie

```
■■■■■■■■■■■■■■■■■■■■■■■
■   «abstract»        ■
■   Utilisateur       ■
■■■■■■■■■■■■■■■■■■■■■■■
■ -id : int           ■
■ -nom : String       ■
■ -username : ...     ■
```

```
■ -role : String    ■
■■■■■■■■■■■■■■■■■■■■■■
■ +login() : bool   ■
■■■■■■■■■■■■■■■■■■■■■■
          ■  (héritage)
■■■■■■■■■■■■■■■■■■■■■■■■■■
■          ■          ■
■■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■■■■■
■ Client  ■ ■Serveuse ■ ■ Cuisinier ■
■■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■■■■■
```

4.4 Relations entre les classes

Relation	Type	Multiplicité	Description
Menu → Plat	Composition	1 à 1..*	Un menu contient plusieurs plats
Commande → LigneCommand	eComposition	1 à 1..*	Une commande comprend des lignes détail
LigneCommande → Plat	Association	1 à 1	Chaque ligne référence un plat
Commande → Facture	Association	1 à 0..1	Une commande peut générer une facture
Client → Commande	Association	1 à 0..*	Un client passe des commandes
Serveuse → Commande	Association	1 à 0..*	Une serveuse gère des commandes

5. Base de données

5.1 Schéma relationnel

```
UTILISATEUR (id PK, nom, prenom, username, password, role, actif)
      |
      |__(id_client)___> COMMANDE (id PK, date_commande, statut,
      |__(id_serveuse)>            num_table, id_client FK, id_serveuse FK)
                                            |
MENU (id PK, nom, description)              |___> LIGNE_COMMANDE (id PK,
  |                                         |           quantite,
  |__(id_menu)_> PLAT (id PK, nom,          |           prix_unitaire,
                       prix, disponible,    |           id_commande FK,
                       id_menu FK)          |<__(id_plat) id_plat FK)
                                            |
                                            |___> FACTURE (id PK,
                                                          montant_total,
NOTIFICATION (id PK, message, date_envoi,                  remise,
              lue, id_commande FK,                         montant_final,
              id_serveuse FK)                              id_commande FK UQ)
```

5.2 Table : utilisateur

Colonne	Type	Contrainte	Description
id	INT	PK, AUTO_INCREMEN	TIdentifiant unique
nom	VARCHAR(50)	NOT NULL	Nom de famille
prenom	VARCHAR(50)	NOT NULL	Prénom
username	VARCHAR(30)	NOT NULL, UNIQUE	Identifiant de connexion
password	VARCHAR(255)	NOT NULL	Mot de passe hashé SHA-256
role	ENUM	NOT NULL	CLIENT / SERVEUSE / CUISINIER
actif	BOOLEAN	DEFAULT TRUE	Compte actif

5.3 Table : commande

Colonne	Type	Contrainte	Description
id	INT	PK, AUTO_INCREMEN	TIdentifiant unique
date_commande	DATETIME	DEFAULT NOW()	Date et heure de création
statut	ENUM	DEFAULT 'EN_ATTEN	EE'N_ATTENTE / EN_COURS / PRETE / SERVIE / ANNULEE
num_table	INT	—	Numéro de table du client
id_client	INT	FK → utilisateur(id)	Client ayant passé la commande
id_serveuse	INT	FK → utilisateur(id)	Serveuse en charge

5.4 Table : ligne_commande

Colonne	Type	Contrainte	Description
id	INT	PK, AUTO_INCREMEN	TIdentifiant unique
quantite	INT	NOT NULL, > 0	Nombre d'exemplaires
prix_unitaire	DECIMAL(8,2)	NOT NULL	Prix au moment de la commande
id_commande	INT	FK, ON DELETE CASC	ACDoEmmande parente
id_plat	INT	FK → plat(id)	Plat commandé

5.5 Table : facture

Colonne	Type	Contrainte	Description
id	INT	PK, AUTO_INCREMEN	TIdentifiant unique
date_facture	DATETIME	DEFAULT NOW()	Date d'émission
montant_total	DECIMAL(10,2)	NOT NULL	Somme brute des lignes
remise	DECIMAL(5,2)	DEFAULT 0.00	Pourcentage de remise
montant_final	DECIMAL(10,2)	NOT NULL	Montant après remise
id_commande	INT	FK, UNIQUE	Commande facturée (1-1)

6. Couche Modèle (Model)

6.1 Classe Utilisateur (abstraite)

```
package model;

public abstract class Utilisateur {
    private int    id;
    private String nom, prenom, username, password, role;

    // Constructeurs, getters, setters...
    public abstract String getNomComplet();
    public String toString() { return "[" + role + "] " + getNomComplet(); }
}
```

Les sous-classes héritent de Utilisateur et ajoutent leurs attributs spécifiques :

Sous-classe	Attribut spécifique	Rôle
Client	numTable : int	Numéro de la table occupée
Serveuse	numBadge : int	Numéro de badge de la serveuse
Cuisinier	specialite : String	Spécialité culinaire

6.2 Cycle de vie d'une commande

```
EN_ATTENTE
    |
    v  (cuisinier commence)
EN_COURS  ■■■
```

7. Couche DAO

7.1 Interface générique IDao

```
package dao;
import java.util.List;

public interface IDao<T> {
    boolean  insert(T  obj);      //  CREATE
    boolean  update(T  obj);      //  UPDATE
    boolean  delete(int id);       // DELETE
    T           findById(int  id);       //  READ  (un  enregistrement)
    List<T>  findAll();            // READ (tous les enregistrements)
}
```

7.2 Implémentations DAO

Classe DAO	Méthodes supplémentaires
UtilisateurDAO	authenticate(username, password)
PlatDAO	findByMenu(idMenu)
CommandeDAO	findByStatut(statut), changerStatut(id, statut)
LigneCommandeDAO	findByCommande(idCommande)
FactureDAO	findByCommande(idCommande)

7.3 Connexion base de données — Singleton

```
package util;
import java.sql.*;

public class DBConnection {
    private       static       final       String       URL       =
        "jdbc:mysql://localhost:3306/restaurant_db?useSSL=false";
    private static final String USER = "root";
    private static final String PWD  = "root";
    private static Connection instance = null;

    public static Connection getInstance() {
        try {
            if (instance == null || instance.isClosed()) {
                Class.forName("com.mysql.cj.jdbc.Driver");
                instance = DriverManager.getConnection(URL, USER, PWD);
            }
        } catch (Exception e) { e.printStackTrace(); }
        return instance;
    }
}
```

8. Couche Contrôleur (Controller)

8.1 AuthController — Authentification

```
public boolean login(String username, String password) {
    utilisateurConnecte = utilisateurDAO.authenticate(username, password);
    return utilisateurConnecte != null;
}
public String getRole() { return utilisateurConnecte.getRole(); }
```

8.2 CommandeController — Cycle de vie

Méthode	Statut résultant	Acteur
creerCommande()	EN_ATTENTE	Client / Serveuse
commencerTraitement()	EN_COURS	Cuisinier
marquerPrete()	PRETE + notification	Cuisinier
marquerServie()	SERVIE	Serveuse
annulerCommande()	ANNULEE	Cuisinier
genererFacture()	Facture créée en BDD	Serveuse

9. Couche Vue (View)

9.1 LoginFrame — Écran de connexion

Interface initiale commune aux 3 rôles. Après validation des identifiants, redirige automatiquement vers le tableau de bord correspondant.

Composant	Type Swing	Rôle
Champ utilisateur	JTextField	Saisie du nom d'utilisateur
Champ mot de passe	JPasswordField	Saisie masquée du mot de passe
Bouton Se connecter	JButton	Déclenchement de l'authentification
Message erreur	JLabel (rouge)	Affichage en cas d'échec

9.2 GestionMenuFrame — Les 5 boutons requis

Bouton	Action
<< Ajouter >>	Ouvre un formulaire JDialog pour créer un nouveau plat
<< Modifier >>	Charge le plat sélectionné dans un formulaire d'édition
<< Supprimer >>	Supprime le plat après confirmation (JOptionPane)
<< Enregistrer >>	Sauvegarde les modifications dans la base de données MySQL
<< Fermer >>	Ferme la fenêtre courante (dispose())

9.3 Tableaux de bord par rôle

Frame	Fonctionnalités disponibles
DashboardClientFrame	Parcourir le menu + Passer une commande
DashboardServeuseFrame	Menu + Commander + Commandes en cours + Reçues + Facture
DashboardCuisinierFrame	Gérer plats (CRUD) + Traiter commandes + Notifications

10. Fonctionnalités par rôle

10.1 Client

1. Se connecter avec username / password
2. Parcourir le menu (liste des plats par catégorie)
3. Sélectionner des plats et valider une commande

10.2 Serveuse

1. Se connecter
2. Parcourir le menu et commander pour un client
3. Visualiser les commandes EN_ATTENTE
4. Visualiser les commandes EN_COURS
5. Générer la facture pour les commandes SERVIES

10.3 Cuisinier

1. Se connecter
2. Ajouter / Modifier / Supprimer des plats du menu
3. Afficher les commandes EN_ATTENTE → Commencer ou Annuler
4. Afficher les commandes EN_COURS → Marquer comme PRETE
5. Envoyer une notification automatique à la serveuse
6. Afficher les commandes SERVIES (historique)

11. Guide d'installation

11.1 Prérequis

- JDK 17 ou supérieur
- MySQL Server 8.0+
- MySQL Workbench (optionnel, recommandé)
- IntelliJ IDEA ou Eclipse
- mysql-connector-j-8.x.x.jar (driver JDBC)

11.2 Étapes

Étape 1 — Créer la base de données

```
-- Dans MySQL Workbench ou CLI :
source /chemin/vers/restaurant_db.sql;
```

Étape 2 — Configurer la connexion

```
// src/util/DBConnection.java
private static final String URL  = "jdbc:mysql://localhost:3306/restaurant_db";
private static final String USER = "root";       // votre utilisateur MySQL
private static final String PWD  = "votre_mdp";  // votre mot de passe MySQL
```

Étape 3 — Ajouter le driver JDBC au classpath

```
# Dans IntelliJ IDEA :
File → Project Structure → Libraries → + → Java → mysql-connector-j.jar
```

Étape 4 — Compiler et lancer

```
# Compilation (ligne de commande)
javac -cp ".:lib/mysql-connector-j.jar" src/**/*.java

# Lancement
java -cp ".:lib/mysql-connector-j.jar:src" view.LoginFrame
```

11.3 Comptes de test

Utilisateur	Mot de passe	Rôle
admin	admin123	Cuisinier
fatma	fatma123	Serveuse
sarra	sarra123	Serveuse
mohamed	client123	Client
amira	amira123	Client

12. Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux de la Programmation Orientée Objet en Java à travers un cas d'usage complet et réaliste. L'implémentation d'une architecture MVC combinée au patron DAO assure une séparation claire et maintenable entre les trois couches de l'application.

12.1 Points clés maîtrisés

- **Héritage et polymorphisme** : hiérarchie Utilisateur -> Client / Serveuse / Cuisinier
- **Interface générique** : IDao avec les 4 opérations CRUD réutilisables pour toutes les entités
- **Patron Singleton** : DBConnection pour une connexion unique et centralisée à la base
- **Architecture MVC** : séparation stricte entre présentation, logique et données
- **Patron DAO** : abstraction complète de l'accès aux données via JDBC
- **Java Swing** : interfaces ergonomiques avec gestion des événements

12.2 Améliorations possibles

- Génération de factures au format PDF avec la bibliothèque iText
- Système de réservation de tables en ligne
- Tableau de bord statistiques : chiffre d'affaires, plats populaires, heures de pointe
- Notification en temps réel via WebSocket entre cuisinier et serveuse
- Migration vers une architecture REST avec Spring Boot et interface web

Documentation technique — Mini-Projet POO — CPI 2 — ISIMG Gabès — Année 2025/2026